데이터구조 필기본

AVL · B-Tree · Hashing · Graph · Shortest Path · MST · String — 핵심 정리노트

Ch 10 AVL Tree — 균형 이진 탐색 트리

자가-균형 BST · Balance Factor · 4가지 회전으로 O(log n) 보장

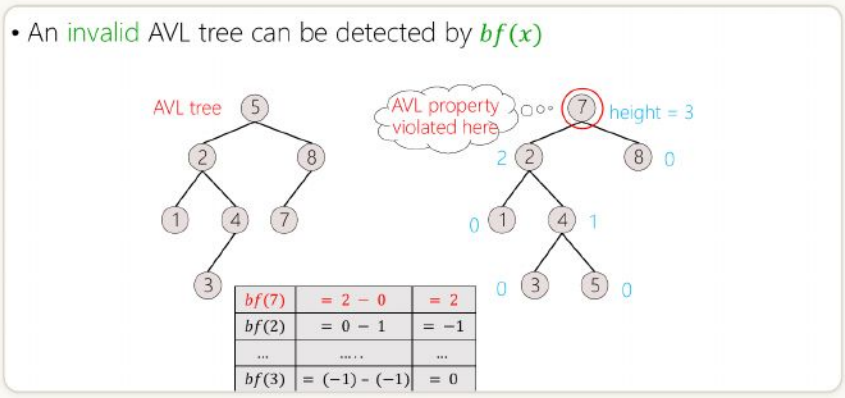
균형 트리 & AVL 정의 p.1-6

왜 균형이 필요한가 — BST 높이가 탐색·삽입·삭제 복잡도를 단독 결정. 정렬된 입력 → 링크드리스트로 퇴화 → O(n) 최악.

- 임의 BST: 평균 O(log n), 최악 O(n) (정렬 입력)
- 균형 BST: 최악과 평균 모두 O(log n)

AVL Tree — 자가-균형 BST. *모든 노드*의 두 서브트리 높이 차 ≤ 1. height-balanced BST.

- Balance Factor — `bf(x) = height(left(x)) − height(right(x))`
- 높이 관례: 잎 `height = 0`, 빈 트리 `height = −1`
- AVL 성질 — 모든 노드 x에 $-1 \le bf(x) \le 1$ 성립
- |bf(x)| = 2인 노드가 *불균형 노드* = pivot 후보



AVL 유효·위반 예시 — bf(7)=2 위반, bf(2)=−1 유효 (p.6)

AVL 삽입 알고리즘 p.8-15

Pivot Node — 새 잎 → 루트 경로에서 bf = ±2가 되는 가장 깊은 노드. 삽입당 pivot은 최대 1개.

pivot = 가장 깊은 불균형 조상

- 경로 바깥 노드는 bf 불변 → pivot에서의 회전 1회로 전체 복구 가능
- Early termination — pivot 회전 후 서브트리 height가 삽입 전 값으로 복귀 → 조상 bf 불변 → 즉시 종료

AVL Insertion

```
// 1단계: BST 삽입
Insert key as leaf (standard BST rule)
// 2단계: 잎→루트 경로 추적
for each node x from leaf to root do
  if −1 ≤ bf(x) ≤ 1 then
    proceed to parent(x)
  else  ⟵ bf(x) = ±2, x is the pivot
    do appropriate rotation at x
    reconnect rotated subtree to parent(x)
    update heights of involved nodes
    return  ⟵ early termination
```

회전 후 서브트리 height = 삽입 이전 값 → 조상 bf 전혀 변하지 않음 → O(log n)

불균형 유형 4가지 p.13-14

분류 기준 — pivot A에서 새 노드까지의 *경로 방향*으로 결정. 4가지만 존재.

- LL — A 왼쪽 자식의 왼쪽 서브트리에 삽입 → 단일 오른쪽 회전
- RR — LL의 거울 → 단일 왼쪽 회전
- LR — A 왼쪽 자식의 오른쪽 서브트리에 삽입 → 이중 회전 RR+LL
- RL — LR의 거울 → 이중 회전 LL+RR

경우	조건(삽입 위치)	회전 유형	회전 수
LL	A.left.left 서브트리	Right rotation at A	1
RR	A.right.right 서브트리	Left rotation at A	1
LR	A.left.right 서브트리	Left at B, Right at A	2
RL	A.right.left 서브트리	Right at B, Left at A	2

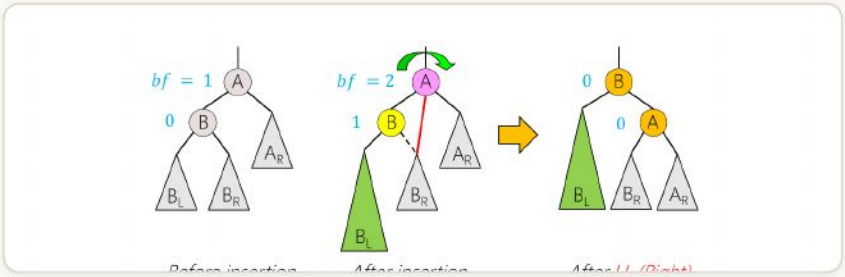
A=pivot, B=A의 더 깊은 자식. 합성 항등식: LR=RR+LL, RL=LL+RR

단일 회전 — LL & RR p.16-21

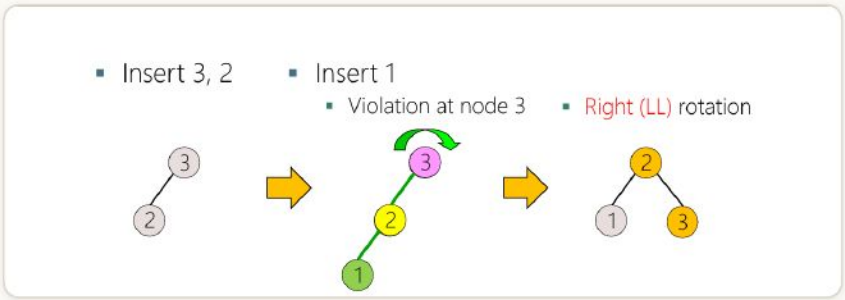
LL — 오른쪽 단일 회전 — pivot `A` (bf=2), 왼쪽 자식 `B` (bf=1). `B`를 새 서브루트로 올리고 `A`를 `B`의 오른쪽 자식으로 내림.

BR이 A.left로 이동

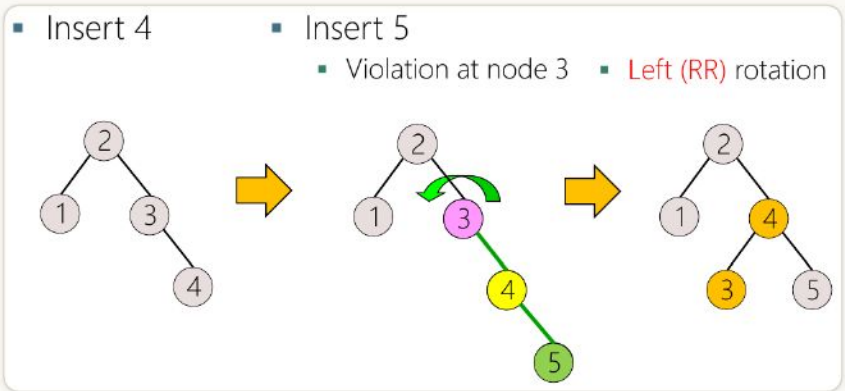
- `BR` (B의 오른쪽 서브트리)가 `A`의 왼쪽 자식으로 이동 — *유일한* 추가 포인터 변경
- 결과: `bf(B)=0`, `bf(A)=0` — 서브트리 height 삽입 전 값으로 복귀
- RR: LL의 거울. `A`의 오른쪽 자식 `B`를 올리는 왼쪽 회전



LL 구조 — Before/After insertion → Right rotation (p.17)



LL 예시 — 3,2,1 삽입 → pivot=3, Right(LL) rotation (p.18)



RR 예시 — Insert 4,5 → violation at node 3 → Left(RR) rotation (p.19)

⚠ **함정** LL은 왼쪽-왼쪽 경로 → 오른쪽 회전. RR은 오른쪽-오른쪽 → 왼쪽 회전. 경우 이름과 회전 방향이 반대라 헷갈림 주의.

이중 회전 — LR & RL p.23-28

왜 단일 회전이 실패하는가 — LR/RL은 새 노드가 *교차 방향* 서브트리에 도착. pivot A에서 단순 회전 시 `bf` 가 반대 방향으로 ±2가 되어 여전히 불균형.

- LR: A(bf=2), B=A.left(bf=–1) — 단일 오른쪽 회전 → B(bf=–2) 새 위반
- 해결: 1차 회전으로 LR 형태를 LL로 변환, 2차 회전으로 마무리

LR 이중 회전 = RR + LL — pivot `A` (bf=2) → 왼쪽 자식 `B` (bf=–1) → `B` 의 오른쪽 자식 `C` .

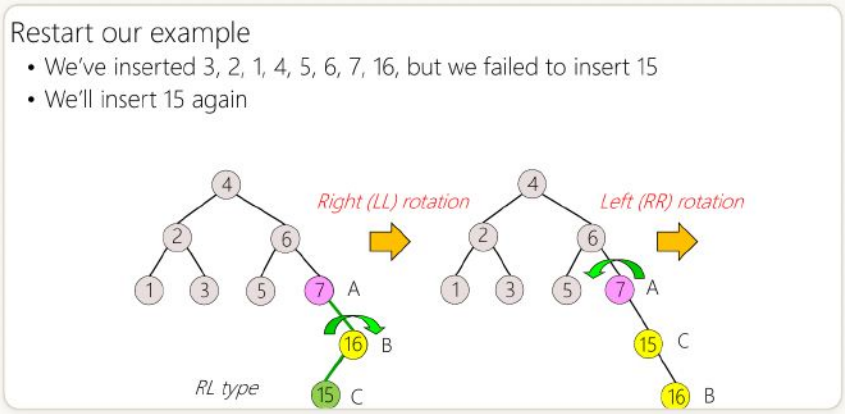
LR=RR+LL
RL=LL+RR

- 1단계 — **B에서 왼쪽(RR) 회전**: C가 올라와 A의 왼쪽 자식, B는 C의 왼쪽 자식으로 하강. A(bf=2) 아직 미해결.
- 2단계 — **A에서 오른쪽(LL) 회전**: C가 서브루트 자리 차지. B는 C.left, A는 C.right로 하강.
- 최종: bf(C)=0, bf(B)=±1, bf(A)=0/–1 — 서브트리 height 복귀
- **RL = LL + RR**: LR의 거울. B=A.right(bf=+1), C=B.left. 1차 LL at B, 2차 RR at A.



① 1차: Left(RR) at B → C 상승, A(bf=2) 미해결 ② 2차: Right(LL) at A → C 서브루트, 균형 복원

LR 이중 회전 전체 과정 (p.25-26)



RL 예시 — Insert 15: A=7, B=16, C=15. RL = LL at A + RR at C (p.27)

LR type-1 vs type-2 — `C` 의 어느 서브트리에 새 노드가 도착했는지에 따라 최종 bf 값이 다름. 그러나 **회전 시퀀스 자체는 동일** — RR+LL (LR), LL+RR (RL).

- type-1: C(bf=–1), 새 노드가 C.left에 → 최종 bf(B)=1, bf(A)=0
- type-2: C(bf=+1), 새 노드가 C.right에 → 최종 bf(B)=0, bf(A)=–1

AVL 삭제 알고리즘 p.36-39

BST 삭제 3가지 경우 복습 — AVL 삭제는 BST 삭제 + 재균형 루프.

- **leaf 노드**: 연결 해제
- **degree-1**: 단일 자식으로 교체
- **degree-2**: 키를 중위 successor(오른쪽 서브트리 최소값)로 교체 → 항상 leaf/degree-1 삭제로 환원

삽입 vs 삭제 재균형의 차이 — 삭제 후 회전이 서브트리 height를 줄일 수 있어 조상에서 연쇄 위반 발생 가능.

삭제=멈추지 않는다

- 삽입: pivot 1회 회전 후 early termination — 최대 1회 회전
- 삭제: early termination 없음 — 루트까지 계속 진행, 최악 O(log n)회 회전

AVL Deletion

```
// 1단계: BST 삭제
Delete node by standard BST rules
// 2단계: 루트까지 경로 추적 (no early stop)
for each node x from deletion point to root do
  if −1 ≤ bf(x) ≤ 1 then
    proceed to parent(x)
  else  ← bf(x) = ±2
    determine rotation type:
      B := child of x with greater height
      C := child of B with greater height
      path A→B→C spells the case (LL/RR/LR/RL)
    do appropriate rotation at x
    continue to parent(x)  ← NO early termination
```

동률 규칙: B의 두 자식 height가 같으면 → single rotation 선택 (double이면 새 위반 생성 가능)

동률(tie) 규칙 — B의 두 자식 height가 같을 때 **반드시 single rotation** 선택.

- single rotation: 서브트리 height가 1 줄어 위반 해소
- double rotation(오선택): 서브트리 하나가 더 깊어져 부모에 새 위반 → 연쇄 폭발

복잡도 & 성능 분석 p.35-35, p.47-48

연산	BST (평균)	BST (최악)	AVL (평균/최악)	회전 수
Search	O(log n)	O(n)	O(log n)	0
Insertion	O(log n)	O(n)	O(log n)	최대 1회
Deletion	O(log n)	O(n)	O(log n)	최대 O(log n)회

AVL은 최악-평균 모두 O(log n) 보장. 삽입 단일회전·삭제 연쇄가 핵심 차이.

AVL vs 일반 BST — 실무 트레이드오프 — AVL이 최악 보장, BST가 평균-케이스 단순성.

- BST 유리: 작은 n (상수 인자 지배), 읽기 위주 워크로드(갱신 드묾), 무작위 입력
- AVL 결정적 승리: **정렬된 입력** — BST O(n) 붕괴, AVL O(log n) 유지
- AVL 비용: 모든 삽입·삭제마다 bf 갱신 + 가능 회전 = 상수 오버헤드

✎ **핵심** 삽입 early termination이 가능한 이유: pivot 회전이 서브트리 **height를 삽입 전 값으로 복원** → 조상 bf 불변. 삭제는 height를 **줄이기** 때문에 조상이 재위반 → 루트까지 진행 필수.

★ **필수암기** bf(x)=h(left)−h(right), AVL 성질: −1≤bf≤1 · 위반=|bf|=2=pivot · LL/RR=단일회전(1회) · LR=RR+LL · RL=LL+RR(2회) · 삽입=early stop(1회 전) · 삭제=루트까지(O(log n)회전) · 전체 O(log n)

Ch 11 B-Tree — 균형 다원 탐색 트리

디스크 I/O 최소화 · Shape+Search Property · 삽입 split · 삭제 borrow/merge

동기 — 왜 다원 탐색 트리인가 p.1-6

디스크 I/O 비용 모델 — 메모리 접근 = *나노초* · 디스크 접근 = **밀리초**. 방문 노드 수 = I/O 횟수.

노드=블록 I/O 단위

- BST: $O(\log_2 n)$ 노드 방문 → $O(\log_2 n)$ I/O
- **다원(m-way) 노드**: 한 노드 = 디스크 블록 1개 → 블록 1회 읽기로 *노드 전체* 획득
- 차수 m=100이면 $\log_{50} 10^9 \approx 4.3$ — 검색당 4~5회 I/O로 끝남

균형 탐색 트리 두 계열 — 저장 매체에 따라 분류. AVL·Red-black·Splay = *메모리 기반*. B-tree 계열 = **디스크 기반**.

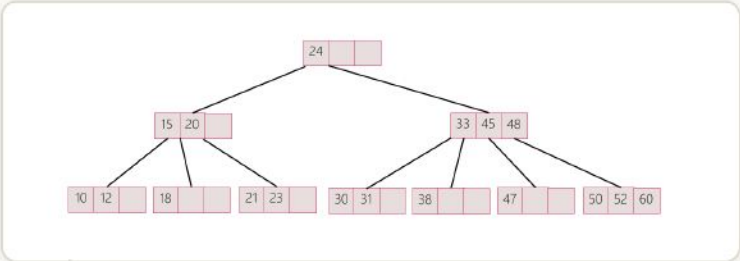
- 메모리 기반: AVL, Red-black, Splay, 2-3, 2-3-4
- 디스크 기반: B-tree, B+-tree, B*-tree, Prefix B-tree

B-Tree 구조 — Shape & Search Property p.3-7

B-Tree (차수 m) 정의 — Shape Property를 만족하는 *m-way* 탐색 트리.

자식 수 [⌈m/2⌉, m]

- **Shape Property** 3조항: ① 루트는 자식 ≥ 2 · ② 루트 외 내부 노드는 자식 ≥ ⌈m/2⌉ · ③ **모든 잎은 동일 깊이**
- **Search Tree Property**: 자식 i의 모든 키 ∈ [부모 k_i, 부모 k_{i+1}) — 각 키가 범위 경계(분리자)로 작동
- 높이 보장: $O(\log_{\lceil m/2 \rceil} n)$ → 검색·삽입·삭제 모두 O(log n)



B-tree order 4 예시 — Shape+Search Property 동시 만족 (p.5)

2-3 트리 (차수 3) — B-tree의 가장 작은 비자명 인스턴스. 내부 노드가 *2 또는 3* 자식.

- 잎 수 범위: $2^h \le n \le 3^h$ → $\log_3 n \le h \le \log_2 n$ = O(log n)
- 자식 수 하한 2가 핵심 — 1이면 사슬 퇴화 가능, **2이면 편향 불가**
- BST는 정렬 입력 시 h=Θ(n) 퇴화 가능; 2-3은 불가 → 최악/평균 모두 O(log n)

Search p.14

2-3 Tree Search(x, node)

```
if x < k1:  return search(x, node.left)
if x = k1:  return node.record(k1)
if k1 < x < k2:  return search(x, node.middle)
if x = k2:  return node.record(k2)
if x > k2:  return search(x, node.right)
```

노드당 최대 2회 비교 후 유일 자식으로 하강. 경로 길이 = h → O(log n). B+ 스타일은 항상 잎까지 내려간다(분리자에서 발견돼도).

⚠ **함정** B+ 트리에서 **내부 노드의 키**는 *분리자*일 뿐 — 실제 레코드는 **잎에만** 존재. 내부에서 키 발견해도 잎까지 내려가야 한다.

Insertion — 분할(Split)과 승격(Promote) p.16-26

삽입 알고리즘 — 찾기 → 직접 삽입 or **분할-승격** 연쇄.

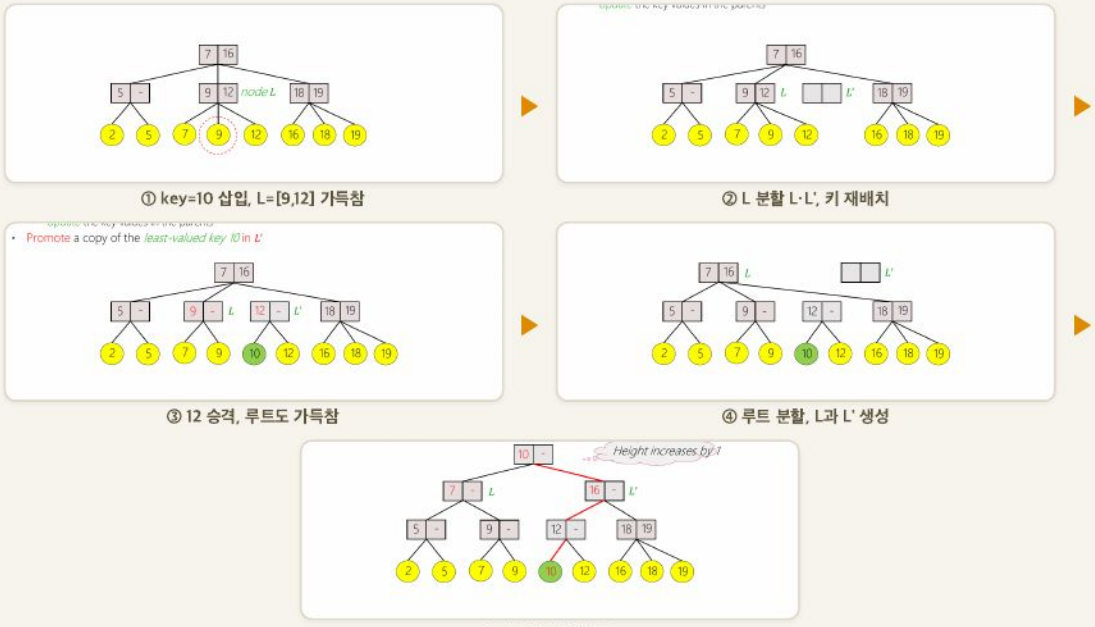
분할=삽입
승격=위로

- Step 1: Search Tree Property로 대상 부모 L 탐색
- Step 2: L이 가득 차지 않았으면 → **직접 삽입** (연쇄 없음)
- Step 3: L이 **가득 찼으면** → L을 L과 L'으로 분할 · 키 균등 재배치 · L'의 최솟값 키를 부모로 **승격**
- 연쇄: 승격 키가 부모도 채우면 같은 분할-승격을 한 레벨 위로 반복
- **루트 분할**: 연쇄가 루트까지 오면 새 루트 생성 → **높이 +1** (트리가 자라는 유일한 경로)

2-3 Tree Insert(key)

```
L ← 부모 노드 찾기(key를 자식으로 두게 될 노드)
if L is not full:  L에 key 직접 삽입
else:
  분할: L → (L, L')  [키를 균등 재배치]
  promote ← L'의 최솟값 키
  Update 분리자 in parent(L)
  if parent(L) is full:  recurse Insert(promote, parent(L))
  else:  parent(L)에 promote 흡수  ← 연쇄 종결
if 루트 분할:  new_root = {promote}; height ← height + 1
```

비용: O(h) = O(log n). 분할당 O(1). 새 루트는 항상 비어있어 연쇄 종결 보장.



① key=10 삽입, L=[9,12] 가득참 ② L 분할 L·L', 키 재배치

③ 12 승격, 루트도 가득참 ④ 루트 분할, L과 L' 생성

⑤ 새 루트 생성, h+1

삽입 key=10 — 이중 연쇄 split·promote 전체 과정 (p.19-26)

✎ **핵심** 트리는 **루트에서만** 자란다. 루트 분할이 유일한 높이 증가 경로. AVL의 회전 없이 Shape Property가 **편향을 구조적으로 차단**.

Deletion — Borrow / Merge p.28-40

삭제 알고리즘 — 3경우 — 부모 N을 찾고 자식 수에 따라 분기.

A: 직접
B: borrow
C: merge

- **Case A** — N이 3자식: 단순 제거 → N은 2자식 유지 · Shape 보존
- **Case B** — N이 2자식 + 형제에 여유: 형제에서 **빌리기(borrow)** + 분리자 재작성
- **Case C** — N이 2자식 + 여유 형제 없음: N의 키를 형제로 이전 + N **병합(merge)** + N 재귀 삭제 → 연쇄 가능

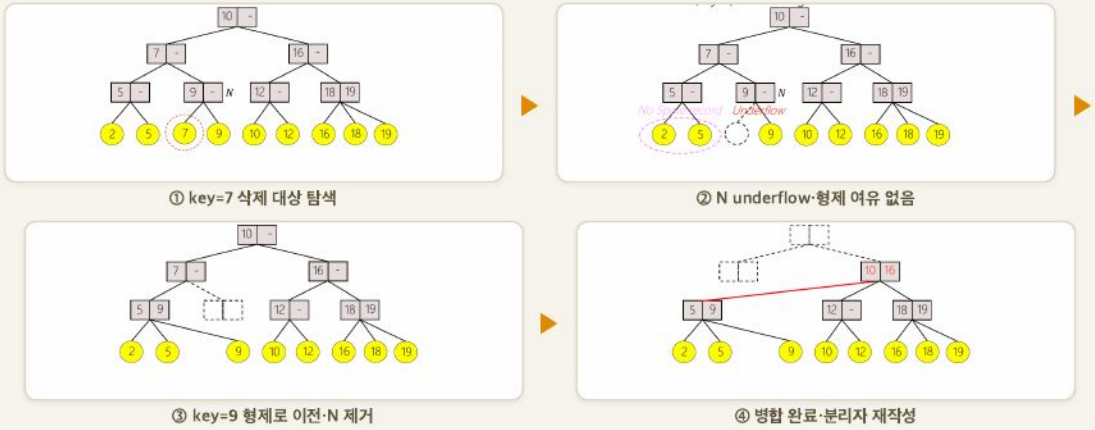
경우	조건	동작	연쇄	높이
Case A	부모 3자식	직접 제거	없음	불변
Case B	부모 2자식·형제 여유	borrow + 분리자 재작성	없음	불변
Case C	부모 2자식·여유 형제 없음	이전 + merge + 재귀 제거	있음	줄 수 있음

삭제 3경우 비교. Case C만 연쇄 → 루트 제거 시 height−1

2-3 Tree Delete(key)

```
N ← key를 자식으로 둔 부모 노드 탐색
if N has 3 children:                ← Case A
  제거 후 분리자 재작성
else:                               ← N has 2 children → Underflow
  if 인접 형제에 여유 자식:          ← Case B
    borrow: 형제에서 키+자식 이전
    분리자 재작성
  else:                             ← Case C
    N의 키를 형제로 이전
    N을 트리에서 제거
    if parent(N) underflows:  recurse Delete(parent(N))
if 루트 자식=1:  루트 제거, height ← height − 1
```

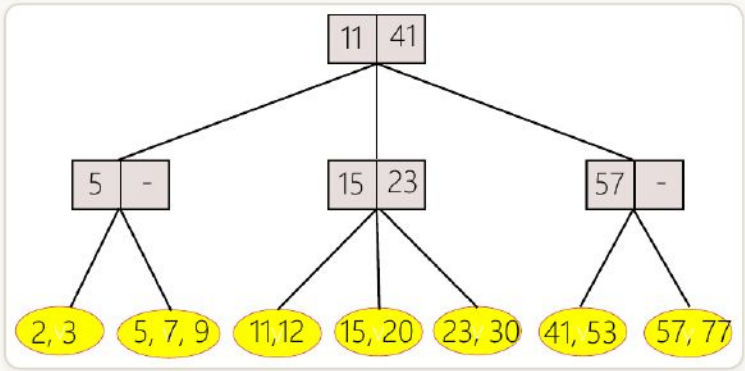
비용: O(h). borrow=O(1). merge 연쇄=레벨당 O(1). 루트에서 종결 보장.



삭제 key=7 — Case C 병합 전체 과정 (p.36-40)

⚠ **함정** 삭제에서 Underflow = 자식이 최소(2) 미만. borrow 시도 → 실패하면 merge. merge 후 부모도 underflow면 연쇄 발생. 루트 자식=1이 되면 루트 제거 → 높이 −1(축소의 유일한 경로).

B+ Tree & 확장 p.7-9



B+ tree order-3 예시 — 내부=분리자, 잎=레코드+연결 (p.10)

B vs B+ vs B* — 세 변형의 핵심 차이.

- **B-tree**: 데이터를 *모든* 노드에 저장 · 키 중복 없음 · 조기 반환 가능
- **B+ tree**: 데이터를 **잎에만** 저장 · 내부 노드 fan-out ↑ · 잎 링크 → *범위 스캔* O(1) 추가
- **B* tree**: B+ 변형 · 노드 채움률 ≥ 2/3 · 오버플로 시 형제 재분배로 분할 지연

	B-tree	B+ tree	B* tree
데이터 위치	모든 노드	잎만	잎만
키 중복	불허	허용(분리자↔잎)	허용
범위 스캔	O(n) DFS	O(k) 잎 링크	O(k) 잎 링크
채움 보장	≥ ⌈m/2⌉/m	≥ ⌈m/2⌉/m	≥ 2m/3
실무 채택	드묾	DB 표준	고밀도 특수 용도

B 계열 비교. 현대 DB는 B+ 채택 — fan-out·범위스캔 우위

✎ 핵심 B+ 표준이 된 이유: ① 내부 노드가 분리자만 → fan-out 극대 ② 잎 링크 → 범위 스캔 선형 ③ 모든 검색 동일 루트-잎 길이 → 비용 예측 ④ 변경이 잎에서만 발생

성능 분석 & 정렬 배열 비교 p.41-43

성장/축소 지점 — 2-3 / B-tree는 항상 루트에서 자라고 루트에서 줄어든다.

- 삽입 연쇄 종점 = 루트 분할 → 새 루트(단일 키)로 종결 → h+1
- 삭제 연쇄 종점 = 루트 자식=1 → 루트 제거 → h−1
- AVL처럼 회전 불필요: Shape Property가 편향 사슬 자체를 *구조적으로 차단*

B-Tree vs 정렬 배열 — 트레이드오프 비교.

- 정렬 배열 *장점*: 공간 효율 ↑ · 이진 검색 메모리 접근 O(log n)
- 정렬 배열 *단점*: 삽입/삭제 O(n) (이동 비용) · 디스크에서 binary search = 무작위 I/O
- B-tree *장점*: 삽입/삭제 O(log n) · 노드=블록으로 디스크 접근 최소화 (m=100 → 10^9개에 4~5 I/O)
- 선택 기준: 정적·읽기전용·메모리 → 정렬 배열 / 동적·디스크·갱신 → B-tree

★ 필수암기 B-tree: Shape(루트≥2자식·비루트≥⌈m/2⌉자식·잎동일깊이)+Search Property · h=O(log n) · 삽입=분할+승격연쇄·루트분할로h+1 · 삭제=Case A(직접)/B(borrow)/C(merge연쇄·루트제거로h−1) · B+는 잎에만데이터·범위스캔O(k)·DB표준 · 2-3트리: $2^h \le n \le 3^h$

Ch 12 Dictionary & Hashing — 딕셔너리와 해싱

key-value 검색 · hash function · 충돌 해결 · 클러스터링 · rehashing

Dictionary ADT & 구현 계열 비교 p.1-9

Dictionary ADT — key-value pair `(k, i)` 의 집합. 핵심 연산 세 가지: insert·find·delete.

- key는 dictionary 내 유일 — 중복 key는 *교체*로 처리
- find(k): key 일치 item 반환, 없으면 miss
- 구현 계열 세 가지 — sequential / hashing / tree indexing

Three approaches to *Search Problem*
- Sequential methods – Sorted list
- *Hashing* method – direct access by *key* values
- *Tree indexing* methods

Data Structure	Worst	Average
Unsorted list	$O(n)$	
Sorted list	$O(\log n)$	
Hash Table	$O(n)$	$O(1)$
Binary Search Tree	$O(n)$	$O(\log n)$
Balanced Search Trees (AVL, 2-3)	$O(\log n)$	

Dictionary 구현 계열 복잡도 비교 (p.3)

복잡도 비교 요약 — Hash table만이 Average O(1)을 달성. Worst는 O(n) (모든 key 충돌).

hash=평균 최속
tree=worst 보장

- Unsorted list: Worst/Avg = O(n) / O(n)
- Sorted list (이진탐색): Worst/Avg = O(log n) / O(log n)
- Hash table: Worst = O(n), Average = O(1)
- AVL / 2-3 tree: Worst/Avg = O(log n) / O(log n) — worst-case 보장

⚠ **함정** Hash O(1)은 *simple uniform hashing* 가정의 기대값. adversarial input이나 편향된 h(·)는 O(n) worst를 유발. *range query*는 hash가 순서 정보를 잃어 tree에 밀림.

Hashing 개요 — hash function `h : key → {0,..,M−1}` 로 slot 주소 직접 계산 — 포인터 추적·비교 없이 배열 index 직접 접근.

- 핵심 압축: 희소 key 도메인(예: 9자리 학번 = 10^9)을 크기 M table로 압축
- 이상적 h = *균일 분산* + *계산 용이* (두 마리 토끼)
- $h(k_1) = h(k_2)$, $k_1 \neq k_2$ → collision (비둘기집 원리로 불가피)

Hash Function 설계 p.10-25

4가지 설계 이슈 — hash table 설계가 답해야 할 독립적이지 않은 네 축.

- ① hash function h 선택
- ② collision 처리 정책
- ③ table 크기 M 결정
- ④ overflow (table 꽉 찼을 때) 처리

네 가지 hash function 계열 — 각 계열의 key→slot 메커니즘.

- Division — `h(x) = x mod M` ; 구현 최단, M 선택에 민감
- Folding — key를 chunk로 나눠 합산 후 mod; 문자열 key에 자연스럽게 일반화
- Mid-square — key^2 계산 후 중간 r bit 추출 ($M=2^r$ 시 최적)
- Digit analysis — 알려진 key 집합에서 편향 digit 제거 후 사용

Division Hashing

```
int hash(int x) {
  return (x % M);
}
```

한 줄 구현. 균일성은 전적으로 M 선택에 의존.

Division — M 선택 규칙 — M이 나쁘면 key 패턴이 residue에 각인되어 분포 편향.

M=짝수→X
M=소수→O

- 짝수 M 금지: h(x)=x mod M이 parity 보존 → 홀수 key는 홀수 slot만, 테이블 절반 낭비
- 홀수지만 작은 소인수 있는 M도 나쁨 — 예: M=651=3×7×31
- 큰 소수 M 권장: gcd(d, M)=1 보장 → 공통 인수 차단, full spread

소수 M이 필요한 이유 — key가 패턴을 가질 때(예: 10, 20, 30, ⋯) 소수 M만이 full spread를 보장.

- Key mod 4 → {2,0,2,0,2} (2종류만) vs Key mod 7 → {3,6,2,4,1} (5종류)
- 일반 규칙: key와 M이 공통 인수 d 공유 시 residue가 gcd 배수로 한정
- 소수 M: 제수가 1과 자기 자신뿐 → gcd 항상 1 → 함정 원천 차단

Folding — 문자열/정수 key — key를 동일 크기 chunk로 분리해 합산 후 mod. 모든 digit이 결과에 기여.

- 예: 123456789, M=10 → chunks 123,456,789 → 합 1368 → 1368 mod 10 = 8
- 단순 합은 *교환 대칭* — 'abc'='cba'. Fix: polynomial rolling hash $\Sigma c_i \cdot B^{t-i}$

Mid-square — key를 제곱 후 중간 r bit 추출. $M=2^r$일 때 정수 key에 탁월.

- 예: $4567^2 = 20{,}857{,}489$ → 중간 2 digit → 57
- 중간 비트는 모든 digit 쌍의 cross-product 반영 → 단일 digit 편향이 분산됨

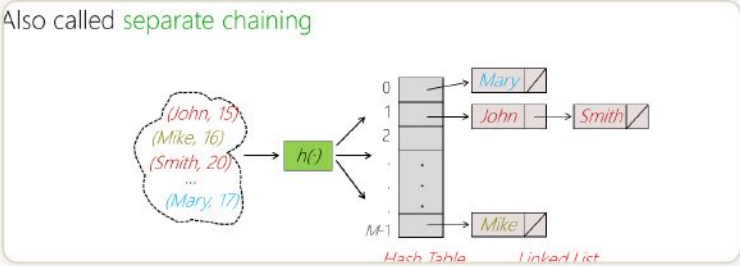
Collision Resolution — 충돌 해결 p.26-36

Static vs Dynamic Hashing — h·M 고정 여부로 분류.

- **Static hashing**: h, M 고정. collision → open(chaining) or closed(open addressing)
- **Dynamic hashing**: M 성장 가능 (extensible, linear hashing). DBMS에 적합
- 용어 주의: **open = separate chaining**, **closed = open addressing** (교과서마다 뒤집힘!)

Separate Chaining (Open Hashing) — 각 slot에 **linked list**를 붙여 충돌 key를 chain에 저장.

- load factor α = n/M = *기대 chain 길이* — α > 1도 허용 (chain 무한 성장)
- **delete**: list 노드 제거, *deleted marker 불필요*
- main memory + in-memory linked list 환경에 적합



Separate Chaining 구조 (p.27)

Separate Chaining — Insert/Find

```
Insert(k, item):
  i := h(k)
  chain[i].prepend((k, item))     ← chain 앞에 추가 (O(1))
Find(k):
  i := h(k)
  for each (key, val) in chain[i] do
    if key == k then return val
  return MISS
```

평균 chain 길이 = α. load factor 낮게 유지 시 O(1) 기대.

Closed Hashing (Open Addressing) — 3-state 모델 — 모든 record를 table 내부에 저장. 외부 chain 없음. 각 slot은 세 상태.

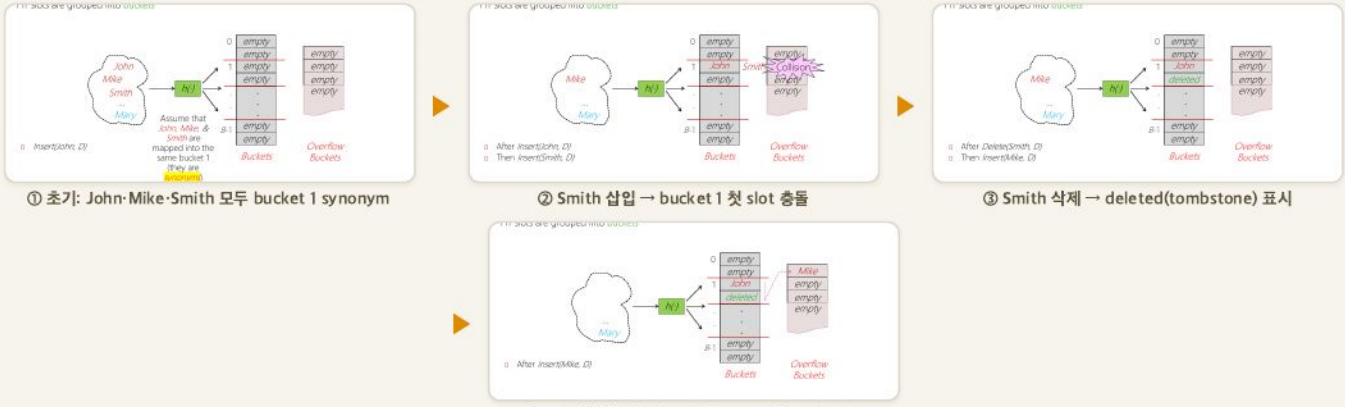
empty=포기
deleted=계속
occupied=비교

- **empty**: 미사용 — search *중단* 신호
- **occupied**: 현재 (k, item) 보유
- **deleted (tombstone)**: 삭제됨 — search *계속* 신호. 없으면 거짓 miss!

⚠ **함정** Delete 후 slot을 **empty**로 바꾸면 그 slot을 넘어 배치된 key가 **search miss** 됨. 반드시 **deleted(tombstone)**으로 표시해야 probe가 계속된다.

Bucket Hashing — M slot을 고정 크기 bucket b로 분할 + 공유 overflow bucket.

- bucket 크기 b = disk block 크기에 맞추면 → 한 번의 disk I/O로 bucket 전체 로드
- 삽입: bucket 내 빈 slot → 없으면 **overflow bucket** (모든 bucket 공유)
- 삭제: slot을 deleted로 표시 → search invariant 유지

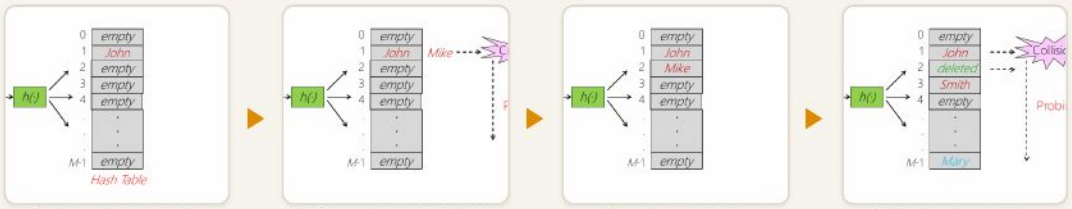


Bucket Hashing 삽입·충돌·삭제·overflow 전체 과정 (p.29-35)

Open Addressing & Probe Function p.37-45

Probe Function 일반 공식 — 충돌 시 다음 slot을 결정하는 규칙. **p(i)의 선택**이 전략을 완전히 결정. p(i)=전략 파라미터

- $h_i(x) = (h(x) + p(i)) \bmod M$, p(0)=0 (첫 시도는 항상 home slot)
- **Linear probing**: p(i) = i
- **Quadratic probing**: p(i) = i^2
- **Double hashing**: p(i) = $i \cdot h_2(x)$



Open Addressing 삽입 전체 과정 — collision·probing·deleted marker (p.38-41)

Linear Probing — p(i) = i — 충돌 시 +1, +2, … 순서로 선형 스캔.

- **Primary clustering**: 연속 점유 run이 그 안에 home이 있는 모든 후속 key를 끌어들임
- Cluster가 길수록 다음 insert가 run을 extend할 가능성 상승 → 양의 피드백
- **장점**: cache locality — 연속 메모리 스캔, production 기본값 많음
- stride c 변형 $p(i)=c \cdot i$: gcd(c,M)=1 필요, 그래도 clustering 해소 불가

Quadratic Probing — p(i) = i^2 — probe gap이 1, 4, 9, 16, … 으로 성장 → primary cluster를 jump over.

- Primary clustering 제거. 그러나 **secondary clustering** 잔존
- 같은 home h(k) 가진 두 key → 동일 probe 수열 공유
- M=2의 거듭제곱이면 모든 slot 방문 실패 → 소수 *M* 또는 $\pm i^2$ 교대 사용

Secondary Clustering — probe 수열이 key가 아닌 *home position만의 함수*일 때 발생.

- $h(k_1)=h(k_2)$인 두 다른 key가 완전히 동일한 probe 수열을 따름
- **Fix**: probe 수열이 **key x 자체에 의존**하게 해야 함 → double hashing
- 계층: primary(최악) → secondary(중간) → 없음(double hashing)

Double Hashing — p(i) = $i \cdot h_2(x)$ — 두 번째 hash function $h_2(x)$ 로 probe 단계를 key별로 modulate.

- $h_i(x) = (h(x) + i \cdot h_2(x)) \bmod M$
- 같은 h(x)라도 $h_2(x)$가 다르면 **다른 probe 수열** → secondary clustering 제거
- 제약: $h_2(x) \neq 0$ (freeze 방지), gcd($h_2(x)$, M)=1 (모든 slot 방문)
- 비용: 연산당 h 평가 *두 회* — secondary clustering 제거의 가격

전략	p(i)	Primary Cluster	Secondary Cluster	비고
Linear	i	있음	없음	cache 친화적
Quadratic	i^2	없음	있음	소수 M 권장
Double hash	$i \cdot h_2(x)$	없음	없음	$h_2 \neq 0$, gcd=1 조건

Open Addressing 전략 비교

Rehashing & Load Factor p.36·p.52-53

Load Factor α = n/M — table 점유 비율. **α가 작을수록** 평균 probe 수가 줄어든다.

- Open addressing: **α < 1** 유지 필수 (α≥1이면 insert loop 미종료)
- Separate chaining: α > 1도 허용 (chain이 무한 성장)
- 통상 임계: **0.5~0.75**에서 rehashing 트리거

Rehashing — α가 임계를 넘으면 더 큰 새 table을 할당하고 *live record만* 재삽입.

O(n) 1회 → amortized O(1)

- tombstone은 **버림** — 재삽입 대상 아님 (load factor 부풀림 해소)
- 1회 비용 O(n), 하지만 n번 insert에 1번꼴 → **amortized O(1)**
- primary clustering + tombstone 누적 두 문제를 동시에 해결

✎ **핵심** Rehashing이 **amortized O(1)**인 이유: rebuild 비용 O(n)을 그 이후 O(n)번의 insert에 분산하면 연산당 O(1). 임계 너무 작으면 rebuild 빈도↑, 너무 크면 probe 길이↑.

Pseudo-Random Probing & Shift-Register p.45-48

Pseudo-Random Probing — 고정 random permutation $\{d_1, d_2, \cdots, d_{M-1}\}$를 probe 수열로 사용.

- $h_i(x) = (h(x) + d_i) \bmod M$ — permutation 공유 필수
- Primary clustering 제거. 그러나 수열이 home position의 함수 → secondary 잔존
- **Shift-register sequence**: $M=2^k$, 상수 k, 시작값 d_1으로 결정적 permutation 생성

Shift-Register Recurrence

```
d₁ := 임의 시작값 ∈ [1, M−1]
for i := 2 to M−1 do
  d := d * 2              ⟵ 1-bit left shift
  if d >= M then d := d − M
  dᵢ := d XOR k           ⟵ 상수 k와 bitwise XOR
```

full-period 수열 = GF(2) 위 primitive polynomial 조건 만족하는 k. M=8, k=3, d_1=5 → 5,1,2,4,3,⋯

성능 요약 & Adversarial Input p.52-53

	Separate Chaining	Open Addressing
저장 위치	table 외부 chain	table 내부
α 범위	> 1 허용	< 1 필수
delete	list 제거 (간단)	tombstone 필요
clustering	없음	linear→primary, quad/dh→secondary/없음
적합	메인메모리, 포인터 cheap	캐시 친화, 연속 메모리

Separate Chaining vs Open Addressing 비교

Adversarial Input & Worst Case — hash table의 평균 O(1)은 *simple uniform hashing* 가정의 기대값.

- Worst case: 모든 key가 같은 slot으로 hash → O(n) probe
- **Hash-flooding 공격**: 의도적으로 충돌 key를 보내 상수 시간을 선형 시간으로 전환
- 완화책: **Universal hashing**(랜덤 family, 공격자가 예측 어려움) 또는 balanced tree

> ★ 필수암기 Dictionary=insert·find·delete · h: key→[0,M-1] · 4이슈(h선택/충돌/M크기/overflow) · M은 큰 소수 · Chaining=외부list·α>1허용·delete간단 · OpenAddressing=내부·α<1·tombstone · Linear=primaryCluster·cache good · Quadratic=secondary · DoubleHash=secondary없음 ($h_2 \neq 0 \cdot gcd=1$) · Rehashing=amortized O(1)·임계0.5~0.75 · Worst=O(n)

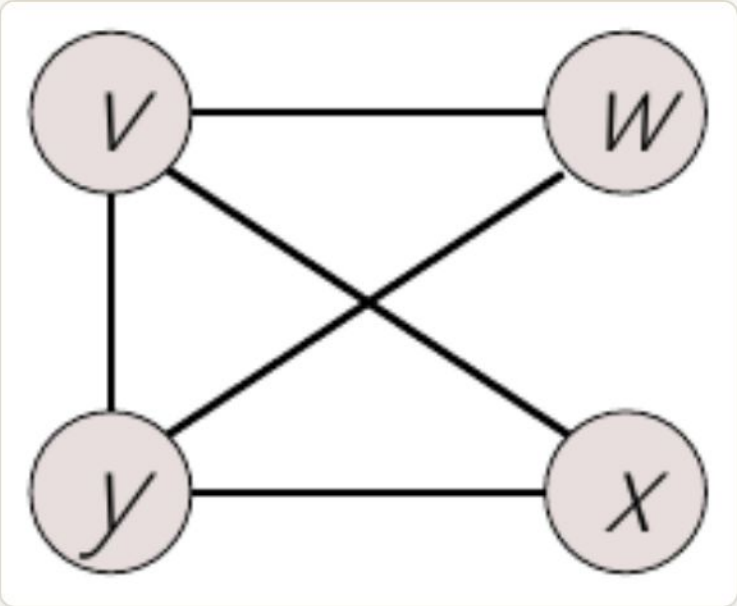
Ch 13 Graph Representations — 그래프 표현

인접 행렬 vs. 인접 리스트 · 공간/시간 trade-off · CSR/CSC

그래프 모델 & 용어 p.1-7

Graph G = (V, E) — 정점 집합 V(|V|=n)와 간선 집합 E(|E|=e)의 순서쌍. 간선 (v,w)는 두 정점을 잇는 쌍.

- **adjacent** — 두 정점이 간선으로 직접 연결 (*정점-정점*). 방향 그래프: v is adjacent **to** w (v→w) / **from** v (←)
- **incident** — 간선이 정점에 닿아 있는 관계 (*정점-간선*). (v,w)는 v와 w에 incident



G = ({v,w,x,y}, {(v,w),(v,y),(w,y),(y,x),(x,v)}) 예제 (p.3)

간선 밀도에 따른 분류 — 간선 수 e가 n에 대해 어떻게 자라는가 — 자료구조 선택의 **기준선**.

- **Complete graph K_n** — 모든 정점 쌍 간선 존재. e = n(n-1)/2
- **Sparse** — `e = O(n)` . 소셜 네트워크·도로망 대부분
- **Dense** — `e = Θ(n²)` . 인접 행렬이 낭비 없는 구간

Path / Cycle / Simple graph — 경로 = 정점 시퀀스를 잇는 간선 시퀀스. *길이* = 간선 수.

- **Simple path** — 반복 정점 없는 경로
- **Cycle** — 시작=끝 정점인 경로 (길이 ≥ 3)
- **Simple graph** 가정 — 두 정점 사이 간선 최대 1개, self-loop 없음

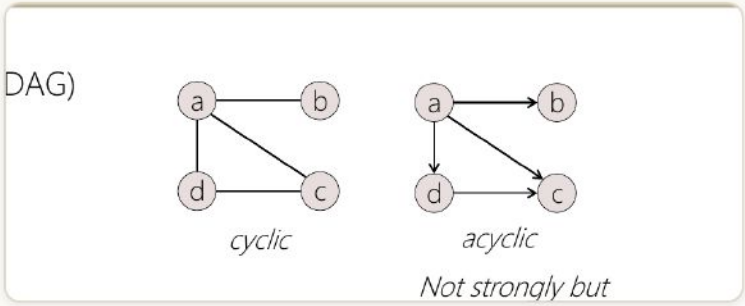
Subgraph vs. Induced subgraph — V_s를 고른 뒤 간선을 어떻게 결정하는가의 차이.

V_s 고정 → E_s 유일 결정

- **Subgraph** G_s — V_s ⊆ V, E_s ⊆ E 자유 선택 (단 양 끝점 ∈ V_s)
- **Induced subgraph** — V_s 지정 시 E_s가 **자동 결정**: E_s = {(v,w)∈E | v,w∈V_s}

Acyclic & Connectedness — 사이클 유무 + 연결성. 방향/무방향에 따라 세 갈래.

- **DAG** (Directed Acyclic Graph) — 방향 + 비순환. 위상 정렬 기반
- **Connected** (무방향) — 임의의 두 정점 사이 경로 존재 ↔ component = 1개
- **Strongly connected** (방향) — 모든 정점 쌍에 **양방향 경로** 존재
- **Weakly connected** (방향) — 방향 제거한 underlying 무방향 그래프가 connected



cyclic vs. acyclic(DAG) · weakly connected 예시 (p.7)

Tree · Spanning Tree · MST p.8-11

Connected Component — 연결된 *maximal subgraph*. G가 connected ↔ component가 정확히 1개.

- 방향 그래프에서는 반드시 **strongly connected component**로 명시

Tree — 다음 동치 조건 중 하나를 만족하는 *무방향 그래프*.

최소 연결 = 최대 비순환

- (a) connected & **acyclic** — 사이클 없는 연결 그래프
- (b) connected & 정점 n개에 **간선 n-1개**
- 간선 하나 추가 → cycle, 간선 하나 제거 → disconnected

Spanning Tree — 연결 그래프 G의 *모든 정점을 포함*하는 트리. V_T = V_G, |E_T| = n-1.

- G에 여분 간선(사이클)이 있으면 신장 트리는 **유일하지 않다**
- 가중치 부여 시: 간선 가중치 합이 최소인 신장 트리 = **MST**

MST (Minimum-Cost Spanning Tree) — 신장 트리들 중 *tree cost* (= 간선 가중치 합) 최소인 것. Kruskal/Prim(Ch.16)의 출발점.

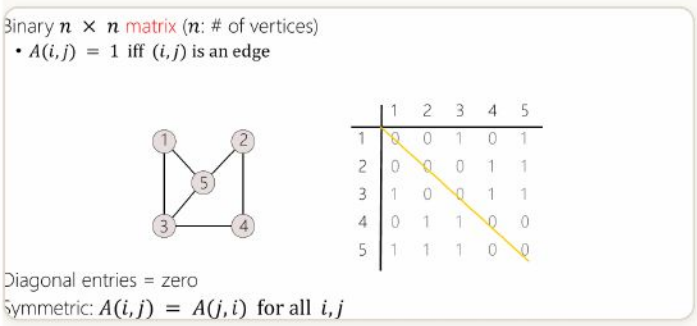
- tree cost 비교: ST-1 cost=16, ST-2 cost=17 → **ST-1이 MST**

Adjacency Matrix — 인접 행렬 p.12-14

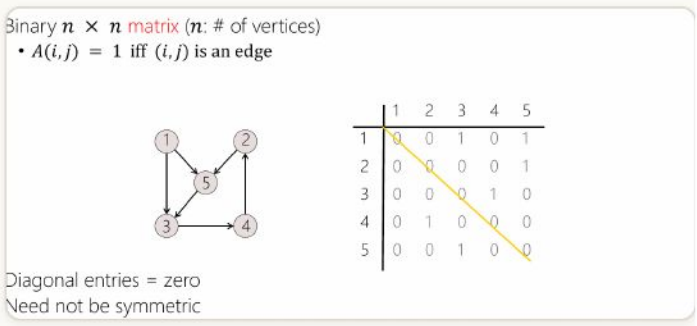
정의 — n×n *binary matrix*. A(i,j) = 1 iff (i,j) is an edge.

A(i,j)=1 iff edge (i,j)

- 공간: **O(V²)** — 정점 1만 개면 셀 1억 개. dense/소규모 그래프에 적합
- 간선 존재 여부 조회: **O(1)** — 행렬 인덱스 직접 접근
- 정점 v의 이웃 순회: **Θ(n)** — 행(또는 열) 전체를 봐야 함



무방향: A(i,j)=A(j,i) 대칭, 대각=0 (p.13)



방향: 비대칭 가능. 행합=out-deg, 열합=in-deg (p.14)

	무방향 Undirected	방향 Directed
대칭성	A(i,j) = A(j,i) 항상	비대칭 가능
대각선	0 (no self-loop)	0 (no self-loop)
행의 합	degree(i)	out-degree(i)
열의 합	degree(i)	in-degree(i)

인접 행렬 무방향 vs. 방향 핵심 성질

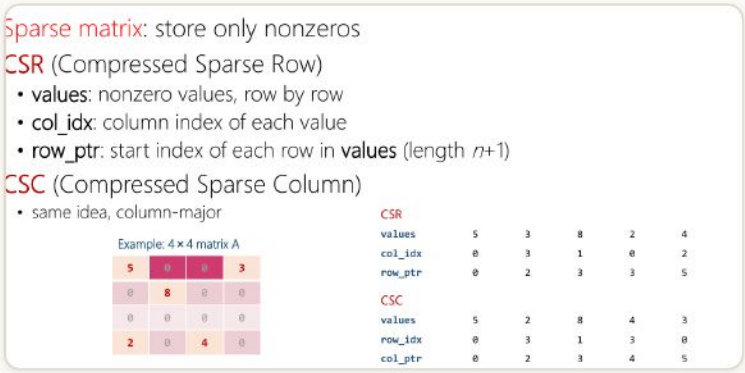
⚠ **함정** 방향 그래프에서 A(i,j)=1이라도 A(j,i)=0일 수 있다. 행합=out-degree, 열합=in-degree — 방향 그래프에서만 의미.

Sparse Matrix & CSR / CSC p.15

CSR (Compressed Sparse Row) — 인접 행렬이 sparse할 때 0을 저장하지 않고 nonzero만 압축. Array 인접 리스트와 *동일한 레이아웃*.

row_ptr 마지막 = 총 nonzero 수

- `values` — nonzero 값, 행 순서로 나열
- `col_idx` — 각 값의 열 인덱스 (values와 같은 길이)
- `row_ptr` — 각 행이 values에서 시작하는 인덱스 (길이 n+1)
- CSC — 같은 아이디어를 column-major로: values, row_idx, col_ptr



4×4 희소 행렬 → CSR/CSC 배열 예제 (p.15)

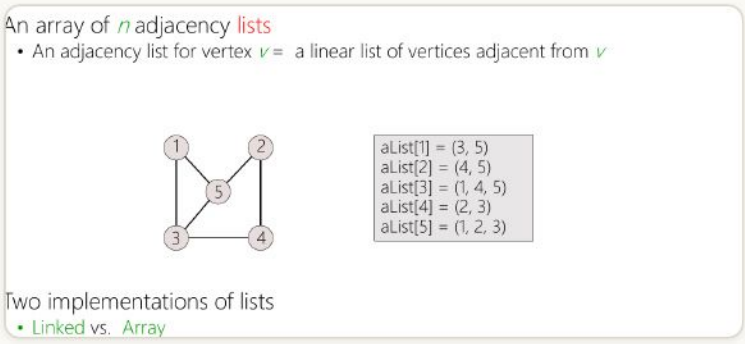
✎ **핵심** CSR row_ptr 해석: 행 i의 nonzero는 values[row_ptr[i] : row_ptr[i+1]]. 마지막 원소 row_ptr[n]= 총 nonzero 수 (sentinel).

Adjacency Lists — 인접 리스트 p.16-19

정의 — 정점마다 하나씩, 총 n개의 인접 리스트. v의 리스트 = *adjacent from v*인 정점들.

공간 O(n+e) vs. O(n²)

- 리스트 길이 = out-degree(v) (방향) / degree(v) (무방향)
- 공간: O(n + e) — sparse 그래프에서 인접 행렬 O(n²) 대비 압도적 절약
- 이웃 순회: O(deg(v)) — 리스트 길이만큼만



aList[1]=(3,5), aList[2]=(4,5), ⋯ 예제 (p.16)

구현 1 — Linked adjacency list — 각 인접 리스트가 *포인터로 연결된 chain*.

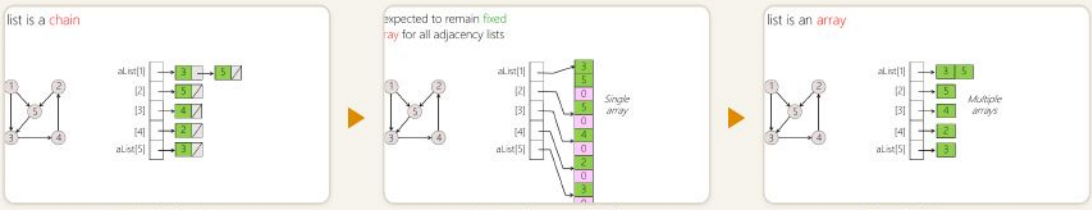
- 장점: 삽입/삭제 용이, 실제 간선 수만큼만 노드 생성
- 단점: 포인터 오버헤드 + cache locality 낮음 (노드 산발)
- 언제: 그래프가 *동적으로 변할 때*

구현 2 — Single array (= CSR) — 모든 인접 리스트를 하나의 연속 배열에 저장. 그래프 고정 가정.

- 장점: 캐시 친화적, 포인터 없음 → 순회 빠름
- 단점: 중간 삽입 시 재배치 필요 — 동적 그래프에 부적합
- 언제: 빌드 후 고정, 대량 순회

구현 3 — Multiple arrays — 정점마다 *독립적인 작은 배열*. Single array와 Linked의 중간.

- Linked보다 접근 빠름(포인터 추종 없음), 길이 추적 쉬움
- Single array보다 연속성 낮음 — 리스트 간 캐시 효율 감소



① Linked chain ② Single array (CSR) ③ Multiple arrays

같은 directed 그래프의 세 가지 인접 리스트 구현 (p.17-19)

표현법 비교 & 선택 기준 p.20 Summary

	인접 행렬	인접 리스트
공간	$O(V^2)$	$O(n + e)$
간선 존재 조회	$O(1)$	$O(deg(v))$
이웃 순회	$\Theta(n)$ 행 전체	$O(deg(v))$
적합	Dense / 조회 빈번	Sparse / 순회 빈번
구현 단순성	단순 (2D 배열)	약간 복잡 (리스트 관리)

같은 그래프, 다른 표현 — Dense면 행렬, Sparse면 리스트

✎ **핵심** sparse 기준: $n=10^4$, $e=10^5$이면 인접 리스트 ≈ 1.1×10^5 셀, 인접 행렬 = 10^8 셀 → 약 *천 배* 차이. 실전(소셜·웹·도로망)은 거의 sparse → 인접 리스트가 기본값.

★ **필수암기** G=(V,E) · adjacent=정점↔정점, incident=정점↔간선 · Complete $e=n(n-1)/2$, Sparse $e=O(n)$, Dense $e=\Theta(n^2)$ · Tree: connected&acyclic, $|E|=n-1$ · 인접행렬: $O(V^2)$공간·$O(1)$조회·대칭(무방향)·행합=out-deg · 인접리스트: $O(n+e)$공간·$O(deg)$조회 · CSR=single-array=row_ptr/col_idx/values

Ch 14 Graph Traversals — 그래프 순회

mark bit로 안전성 확보 · 스택(DFS) vs 큐(BFS) 선택이 순회 성격 결정

Graph Traversal 개요 p.2-4

Graph Traversal / Search — 그래프의 모든 정점을 체계적으로 한 번씩 방문. 트리 순회의 일반화.

- 두 까다로운 경우: not connected (일부 정점 미도달) / cycle (무한 루프)
- 해결책 → mark bit (visited flag)를 정점마다 유지: 이미 방문한 정점 재진입 차단
- 트리 순회는 acyclic + unique path → mark 불필요; 그래프는 두 속성 모두 없음 → *필수*
- 대표 알고리즘: DFS (재귀 스택) / BFS (명시적 큐)

Depth-First Search (DFS) p.5-13

DFS 정의 — 각 자식을 **가능한 한 깊게** 탐색 후 backtrack. 트리 preorder traversal의 그래프 버전.

- 구현: *재귀(recursion stack)* 기반 — 호출 스택이 현재 경로를 유지
- backtrack = unvisited 인접이 없으면 호출 스택을 거슬러 올라가 가장 최근의 unvisited 형제로

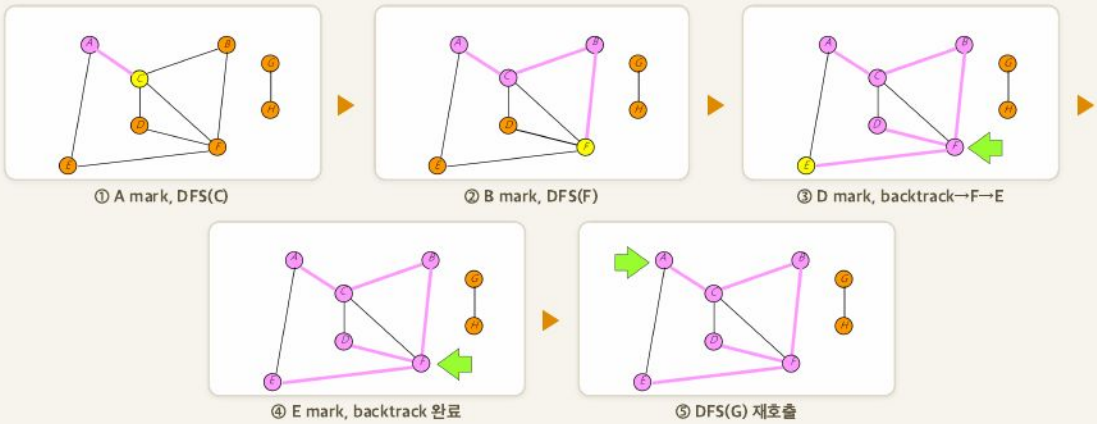
DFS(v)

```
mark[v] := visited;
for (each unvisited vertex w adjacent from v)
  DFS(w);
```

mark 후 unvisited 인접을 재귀 — 두 줄이 DFS의 전부.

DFS 전체 과정 예제 (A~H) — A에서 출발. A→C→B→F→D backtrack→E backtrack→A 완료. G·H는 별도 호출.

- 호출 스택 절정: `DFS(A)→DFS(C)→DFS(B)→DFS(F)→DFS(D)`
- G, H는 A의 DFS 완료 후 *새 DFS(G) 호출* → spanning forest의 두 번째 트리



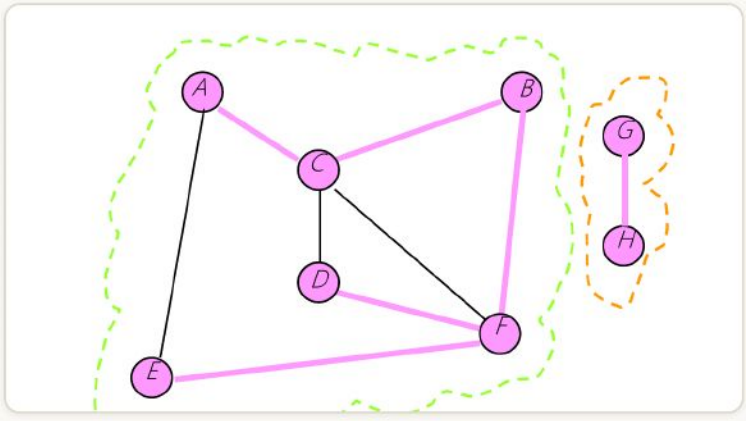
① A mark, DFS(C) ② B mark, DFS(F) ③ D mark, backtrack→F→E

④ E mark, backtrack 완료 ⑤ DFS(G) 재호출

DFS 전체 과정 — mark·재귀·backtrack 흐름 (p.6-12)

Depth-First Spanning Forest — pink edges = tree edge 집합 → spanning forest. 연결 성분 하나 = 트리 한 그루.

- tree edge 수 = 전체 정점 수 − DFS tree 수 (비순환 + 전정점 포함 = 트리 정의)
- forest의 트리 개수 = connected component 개수



DFS spanning forest — 2개 성분(녹색·주황 점선) (p.13)